Neural Networks

Technical reference

The general model of a neural network, learning tasks and their error, data representation, and threshold logic units with the delta rule, multi-layer perceptrons with universal approximation, activation functions, training and depth, autoencoders, radial basis function networks, learning vector quantization and self-organizing maps, with proofs. Each chapter says which site note explains it; the last chapter collects the formulas.

Companion to the notes at https://lorenzoliuzzo.github.io/notes/

CONTENTS

1 The general model 1
1.1 Networks, units and connections 2
1.2 Acyclic networks 2
2 Learning tasks 3
2.1 Fixed and free tasks 3
2.2 The error as maximum likelihood 3
3 Data representation 4
4 Threshold logic units 4
4.1 The unit and its hyperplane 4
4.2 Linear separability 5
4.3 Two layers of threshold units 5
4.4 The delta rule 6
5 Multi-layer perceptrons 7
5.1 Layered networks 7
5.2 Universal approximation 7
6 Activation functions 8
7 Training 9
7.1 Gradient descent 9
7.2 Variants 9
7.3 Complexity 10
8 Depth 10
9 Autoencoders 11
10 Radial basis function networks 12
11 Learning vector quantization 14
12 Self-organizing maps 15
13 Summary of formulas 16
14 References 17

1 The general model

In the notes: Introduction to Neural Networks

1.1 Networks, units and connections

DEFINITION Neural network A neural network is a directed graph $G = (U, C)$ with a finite set U of *units* and a set $C \subseteq U \times U$ of *connections*, together with a weight $w_{uv} \in \mathbb{R}$ for every connection $(v, u) \in C$. The set of predecessors of a unit u is $\mathrm{pred}(u) = \{v : (v, u) \in C\}$. The units are partitioned into the input units U_{in} and output units U_{out}, both non-empty and not necessarily disjoint, and the hidden units $U_{\mathrm{hid}} = U \setminus (U_{\mathrm{in}} \cup U_{\mathrm{out}})$.

Every unit u has three real state variables, the *network input* net_u, the *activation* act_u and the *output* out_u, and every input unit has a fourth, the *external input* ext_u. Every unit has three functions, with θ_u the vector of its own parameters (for example a threshold):

$$\mathrm{net}_u = f_{\mathrm{net}}^{(u)}(w_u, \mathrm{in}_u), \quad \mathrm{act}_u = f_{\mathrm{act}}^{(u)}(\mathrm{net}_u, \theta_u), \quad \mathrm{out}_u = f_{\mathrm{out}}^{(u)}(\mathrm{act}_u), \tag{1}$$

where $w_u = (w_{uv})_{v \in \mathrm{pred}(u)}$ and $\mathrm{in}_u = (\mathrm{out}_v)_{v \in \mathrm{pred}(u)}$. For an input unit, $\mathrm{net}_u = \mathrm{ext}_u$.

DEFINITION Network input function The function $f_{\mathrm{net}}^{(u)}$ of a unit. The standard choice is the weighted sum $\mathrm{net}_u = \sum_{v \in \mathrm{pred}(u)} w_{uv}\,\mathrm{out}_v$; radial basis function units use a distance $d(w_u, \mathrm{in}_u)$ instead. The activation and output functions are, for the standard threshold unit, a step at θ_u and the identity.

Architectures differ by the graph and by which of the three functions they change: a distance replaces the sum in radial basis function networks, a winner-takes-all rule replaces the identity output in learning vector quantization, and in a Hopfield network $U_{\mathrm{in}} = U_{\mathrm{out}} = U$ and $U_{\mathrm{hid}} = \varnothing$.

1.2 Acyclic networks

DEFINITION Feed-forward network A neural network whose graph has no directed cycle. A network with a cycle is called recurrent.

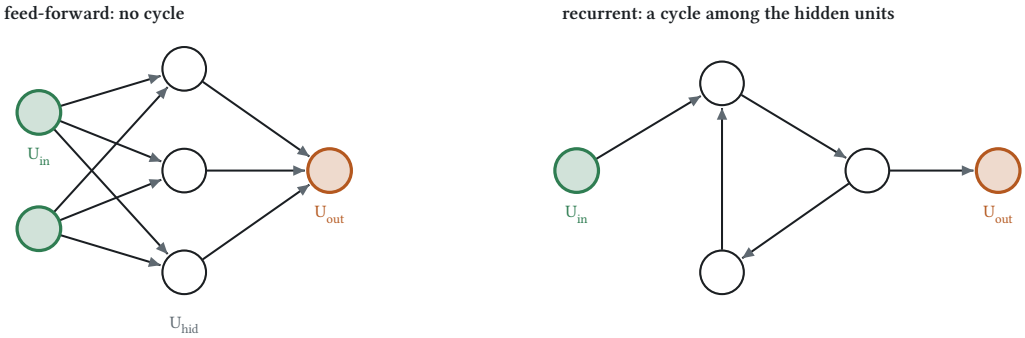
LEMMA A finite directed graph is acyclic if and only if its vertices can be numbered $u_1, \ldots, u_n$ so that $(u_j, u_i) \in C$ implies $j < i$ (a *topological order*).

Proof. If such a numbering exists, a cycle $v_1 \to v_2 \to \ldots \to v_1$ would give an increasing sequence of numbers returning to its start, which is impossible. Conversely, let G be acyclic. Some vertex has no predecessor: otherwise, starting anywhere and repeatedly moving to a predecessor produces, after $n + 1$ steps, a repeated vertex and therefore a cycle. Give that vertex the number 1, remove it (the rest is still acyclic) and number the remaining vertices by induction. □

THEOREM Evaluation of a feed-forward network Let the external inputs be fixed and the units be taken in a topological order. Computing for each unit in turn net_u, act_u and out_u determines every output after $|U|$ unit evaluations, and the result does not depend on which topological order is used.

Proof. By induction on the position i in the order: unit u_i depends only on the weights, on θ_{u_i} and on the outputs of its predecessors, all of which have position below i and are already determined (an input unit depends only on its external input). So the value of out_{u_i} is a function of the data alone, defined by the same recursion for every topological order. □

Remark. A recurrent network has no such order. After the input phase, with the external input switched off, units are recomputed whenever one of their inputs has changed, until the outputs are stable or a bound on the number of recomputations is reached. Both the outcome and whether a stable state exists can depend on the update order. For example, two units u, v with $\mathrm{out}_u := \mathrm{out}_v$ and $\mathrm{out}_v := 1 - \mathrm{out}_u$ and outputs in $\{0, 1\}$ pass through $(0,0) \to (0,1) \to (1,1) \to (1,0) \to (0,0)$ and never stabilize.



Left: one pass in topological order computes everything. Right: the outputs must be recomputed until they stop changing.

Figure 1: An acyclic graph (left) and a graph with a cycle (right). Only the first can be evaluated by a single pass in topological order.

2 Learning tasks

In the notes: Introduction to Neural Networks

2.1 Fixed and free tasks

DEFINITION Fixed learning task For a network with input units $u_1, \dots, u_n$ and output units $v_1, \dots, v_m$, a fixed learning task is a finite set $L = \{(\iota^{(l)}, o^{(l)})\}_{l=1}^{N}$ of training patterns, each an input vector $\iota^{(l)} \in \mathbb{R}^n$ (the external inputs) and a desired output vector $o^{(l)} \in \mathbb{R}^m$. The task is solved if for every l the network, given $\iota^{(l)}$, computes $\text{out}_{v_k} = o_k^{(l)}$ for all k. It is the supervised learning task.

DEFINITION Free learning task A free learning task is a finite set $L = \{\iota^{(l)}\}$ of input vectors with no desired outputs. The outputs are chosen by the learning method, under the principle that similar inputs should produce similar outputs.

DEFINITION Sum of squared errors The error of a network on a fixed learning task is

$$e = \sum_{l=1}^{N} \sum_{k=1}^{m} \left(o_k^{(l)} - \text{out}_{v_k}^{(l)} \right)^2, \tag{2}$$

where $\text{out}_{v_k}^{(l)}$ is the output of v_k for input $\iota^{(l)}$. It is non-negative, and it is zero exactly when the task is solved.

2.2 The error as maximum likelihood

THEOREM Squared error and Gaussian noise Suppose the desired outputs satisfy $o^{(l)} = y(\iota^{(l)}; w) + \varepsilon^{(l)}$, where $y(\cdot; w)$ is the function computed by the network with parameters w and the $\varepsilon^{(l)}$ are independent with distribution $\mathcal{N}(0, \sigma^2 I_m)$. Then the log-likelihood of the data is

$$\ell(w, \sigma^2) = -\frac{e(w)}{2\sigma^2} - \frac{Nm}{2} \log(2\pi\sigma^2). \tag{3}$$

For every fixed σ^2 it is maximized by the w that minimizes e, and the maximum likelihood estimate of the variance is $\hat{\sigma}^2 = e(\hat{w})/(Nm)$.

Proof. By independence the density of the data is $\prod_{l,k} (2\pi\sigma^2)^{-1/2} \exp\left(-\left(o_k^{(l)} - y_{k(\iota^{(l)};w)}\right)^2/(2\sigma^2)\right)$; taking logarithms gives the stated ℓ. The first term is a decreasing function of $e(w)$ for fixed σ^2, which gives the claim about w. Setting $\partial\ell/\partial\sigma^2 = e/(2\sigma^4) - Nm/(2\sigma^2) = 0$ gives $\hat\sigma^2 = e/(Nm)$. □

Remark. For class labels the Gaussian model is replaced by a Bernoulli or categorical one, whose negative log-likelihood is the cross-entropy; see Bishop (2006), section 5.2.

3 Data representation

In the notes: Introduction to Neural Networks

DEFINITION **Standardization** For an input with values $x_1, \dots, x_n$, mean $\bar x = 1/n \sum_i x_i$ and corrected standard deviation $s = \sqrt{1/(n-1)\sum_i (x_i - \bar x)^2} > 0$, the standardized values are $x_i' = (x_i - \bar x)/s$.

THEOREM The standardized values have mean 0 and corrected standard deviation 1.

Proof. $1/n \sum_i x_i' = (\bar x - \bar x)/s = 0$ and $1/(n-1)\sum_i x_i'^2 = 1/s^2 \cdot 1/(n-1)\sum_i (x_i - \bar x)^2 = 1$. □

The mean and s are computed on the training data and the same numbers are applied to validation and test data.

DEFINITION **One-hot encoding** A variable with symbols $a_1, \dots, a_N$ is coded by N input units; symbol a_i becomes the vector $e_i \in \{0,1\}^N$ with a 1 in position i and 0 elsewhere (called 1-in-N encoding in the lectures).

THEOREM For $i \neq j$, $\|e_i - e_j\|_2 = \sqrt{2}$. In contrast, coding the symbols by integers z_i gives distances $|z_i - z_j|$ that differ between pairs, and $|z_1 - z_3| = |z_1 - z_2| + |z_2 - z_3|$ whenever z_2 lies between z_1 and z_3.

Proof. $e_i - e_j$ has a 1 in position i, a -1 in position j and zeros elsewhere, so its squared norm is 2. The statement on integer codes is the additivity of distance on a line. □

Remark. Standardization and decorrelation of the inputs are recommended for gradient-based training by LeCun et al. (1998); the argument is about the conditioning of the error surface and is not proved here.

4 Threshold logic units

In the notes: Threshold Logic Units

4.1 The unit and its hyperplane

DEFINITION **Threshold logic unit** A threshold logic unit (TLU, McCulloch-Pitts neuron) with n inputs has weights $w \in \mathbb{R}^n$ and a threshold $\theta \in \mathbb{R}$. It is the unit with net $= w \cdot x$, step activation at θ and identity output:

$$y(x) = \begin{cases} 1 & \text{if } w \cdot x \geq \theta \\ 0 & \text{otherwise.} \end{cases} \tag{4}$$

Its boundary is the hyperplane $H = \{x : w \cdot x = \theta\}$ with normal w, at distance $\theta/\|w\|$ from the origin when $w \neq 0$. The unit outputs 1 on H and on the open half-space where $w \cdot x - \theta > 0$.

EXAMPLE **Conjunction and implication** $w = (1,1), \theta = 3/2$ gives $y = 1$ exactly at $(1,1)$, since $w \cdot x \in \{0,1,1,2\}$ for the four Boolean inputs. $w = (-2,2), \theta = -1$ gives $w \cdot x \in \{0,2,-2,0\}$ at $(0,0),(0,1),(1,0),(1,1)$ and so $y = 0$ only at $(1,0)$: this is $x_1 \to x_2$.

4.2 Linear separability

DEFINITION **Linear separability** Two finite sets $A_0, A_1 \subset \mathbb{R}^n$ are linearly separable if there are $w \in \mathbb{R}^n$ and $\theta \in \mathbb{R}$ with $w \cdot x < \theta$ for all $x \in A_0$ and $w \cdot x \geq \theta$ for all $x \in A_1$. A Boolean function is computed by a single TLU if and only if the inputs with value 0 and those with value 1 are linearly separable.

THEOREM **Convex hull criterion** Two finite sets $A_0, A_1 \subset \mathbb{R}^n$ are linearly separable if and only if their convex hulls are disjoint, $\operatorname{conv}(A_0) \cap \operatorname{conv}(A_1) = \varnothing$.

Proof. ($\Rightarrow$) If $w \cdot x \geq \theta$ on A_1, then it holds on $\operatorname{conv}(A_1)$, because $w \cdot \left(\sum_i \lambda_i x_i\right) = \sum_i \lambda_i w \cdot x_i$ for $\lambda_i \geq 0$, $\sum_i \lambda_i = 1$. Likewise $w \cdot x < \theta$ on $\operatorname{conv}(A_0)$. A common point would satisfy both.

($\Leftarrow$) The hulls K_1, K_0 are compact and convex, so $\|p - q\|$ attains its minimum d on $K_1 \times K_0$ at some (p, q), and $d > 0$ because the hulls are disjoint. Put $w = p - q$. For $p' \in K_1$ the point $p + t(p' - p)$ lies in K_1 for $t \in [0,1]$, and $\|p + t(p' - p) - q\|^2 = d^2 + 2tw \cdot (p' - p) + t^2\|p' - p\|^2$ cannot fall below d^2 for small $t > 0$; hence $w \cdot (p' - p) \geq 0$, that is $w \cdot p' \geq w \cdot p$. In the same way $w \cdot q' \leq w \cdot q$ for $q' \in K_0$. Since $w \cdot p - w \cdot q = d^2 > 0$, the threshold $\theta = (w \cdot p + w \cdot q)/2$ gives $w \cdot p' > \theta$ on K_1 and $w \cdot q' < \theta$ on K_0. □

Remark. For the biimplication $x_1 \leftrightarrow x_2$ the hulls of $A_1 = \{(0,0),(1,1)\}$ and $A_0 = \{(0,1),(1,0)\}$ are the two diagonals of the unit square, which meet at $(1/2, 1/2)$. So no TLU computes it, and the same argument excludes the exclusive or. Of the 16 Boolean functions of two inputs, the other 14 are separable: the constants, the 8 functions that are true at exactly one or exactly three corners (a corner of the square is an extreme point), and the 4 that are true on one edge. In general the number of separable functions of n inputs is $2, 4, 14, 104, 1882, 94572$ for $n = 0, \ldots, 5$ (OEIS A000609), a vanishing fraction of 2^{2^n}.

4.3 Two layers of threshold units

THEOREM **Boolean functions by two layers** Every Boolean function $f : \{0,1\}^n \to \{0,1\}$ is computed by a network of TLUs with one hidden layer of at most 2^n units.

Proof. For $a \in \{0,1\}^n$ let the unit h_a have $w_i = 2a_i - 1$ and $\theta = \sum_i a_i$. Then

$$w \cdot x = \sum_{i : a_i = 1} x_i - \sum_{i : a_i = 0} x_i \leq \sum_i a_i, \tag{5}$$

with equality if and only if $x_i = 1$ wherever $a_i = 1$ and $x_i = 0$ wherever $a_i = 0$, that is if and only if $x = a$. So $h_a(x) = 1$ exactly at $x = a$. The output unit has weight 1 from each h_a with $f(a) = 1$ and threshold 1: it is the disjunction of those h_a, and equals f. □

EXAMPLE **Biimplication** $h_1 = x_1 \wedge x_2$ ($w = (1,1)$, $\theta = 3/2$), $h_2 = \neg x_1 \wedge \neg x_2$ ($w = (-1,-1)$, $\theta = -1/2$) and $y = h_1 \vee h_2$ ($w = (1,1)$, $\theta = 1/2$). The inputs $(0,0),(0,1),(1,0),(1,1)$ map to $(h_1, h_2) = (0,1),(0,0),(0,0),(1,0)$; the points with $y = 1$ and $y = 0$ are separated by $h_1 + h_2 = 1/2$.

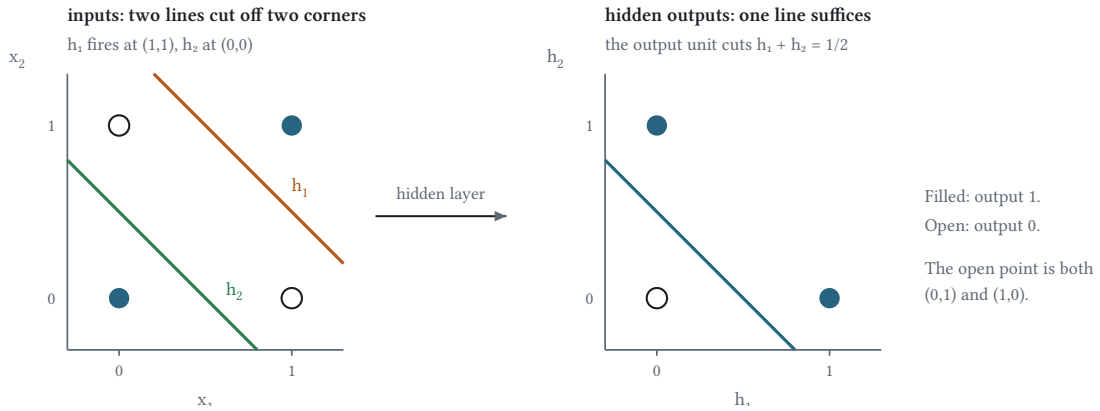


Figure 2: Biimplication by a hidden layer. Left: the two hidden units cut off the corners $(1,1)$ and $(0,0)$ of the input square. Right: in the hidden coordinates the inputs $(0,1)$ and $(1,0)$ coincide at the open point, and the output unit separates it from the two filled points.

4.4 The delta rule

DEFINITION Delta rule Let (x, o) be a training pattern with $o \in \{0,1\}$ and y the output of a TLU. If $y \neq o$, the weights and threshold are changed to

$$\theta \leftarrow \theta - \eta(o - y), \quad w_i \leftarrow w_i + \eta(o - y)x_i, \tag{6}$$

with learning rate $\eta > 0$. If $y = o$ nothing changes. *Online* training applies the change after each pattern; *batch* training sums the changes over one pass through all patterns (an epoch) and applies the sum once. Training stops after an epoch without a change.

Remark. The rule is gradient descent on the modified error $e = \sum_{\text{wrong patterns}} |w \cdot x - \theta|$: for $o = 1$, $y = 0$ the term is $\theta - w \cdot x$ with gradient $(-x, +1)$ in (w, θ), for $o = 0$, $y = 1$ it is $w \cdot x - \theta$ with gradient $(x, -1)$. The count of wrong outputs itself is constant on plateaus, with zero gradient almost everywhere. After one correction $w \cdot x - \theta$ changes by $\eta(o - y)(\|x\|^2 + 1)$, in the direction of $o - y$.

THEOREM Convergence of the delta rule Let $L = \{(x^{(l)}, o^{(l)})\}_{l=1}^{m}$ be training patterns with $o^{(l)} \in \{0,1\}$. If the inputs with $o = 0$ and those with $o = 1$ are linearly separable, then online training, in any order of presentation, and batch training both stop after finitely many corrections, and the final (w, θ) classifies every pattern correctly. If they are not separable, training does not stop.

Proof. Put $z^{(l)} = (x^{(l)}, -1)$, $v = (w, \theta)$ and $s_l = 2o^{(l)} - 1 \in \{-1, +1\}$. Then $y = 1$ iff $v \cdot z \geq 0$; a pattern is misclassified exactly when $s_l(v \cdot z^{(l)}) \leq 0$ (for $o = 0$ the case $v \cdot z = 0$ is an error because the unit outputs 1); and a correction is $v \leftarrow v + \eta s_l z^{(l)}$, because $o - y = s_l$ on an error. Let $R = \max_l \|z^{(l)}\|$.

Take a separating (w^*, θ^*). Lowering θ^* by half the smallest value of $\theta^* - w^* \cdot x$ over the patterns with $o = 0$ gives v^* and $\gamma > 0$ with $s_l(v^* \cdot z^{(l)}) \geq \gamma$ for all l.

Online. After k corrections, starting from v_0: each adds at least $\eta\gamma$ to $v \cdot v^*$, so $v_k \cdot v^* \geq v_0 \cdot v^* + k\eta\gamma$. Each satisfies $\|v + \eta s z\|^2 = \|v\|^2 + 2\eta s(v \cdot z) + \eta^2\|z\|^2 \leq \|v\|^2 + \eta^2 R^2$, so $\|v_k\|^2 \leq \|v_0\|^2 + k\eta^2 R^2$. By Cauchy-Schwarz $v_0 \cdot v^* + k\eta\gamma \leq \|v^*\|\sqrt{\|v_0\|^2 + k\eta^2 R^2}$. The left side grows linearly in k and the right like $\sqrt{k}$, so k is bounded (for $v_0 = 0$, $k \leq (R\|v^*\|/\gamma)^2$). After the last correction an epoch with no correction follows and training stops.

Batch. At v let E be the nonempty set of misclassified patterns and $\Delta v = \eta \sum_{l \in E} s_l z^{(l)}$. Then $\Delta v \cdot v^* \geq \eta\gamma\,|E| \geq \eta\gamma$, and $\|v + \Delta v\|^2 \leq \|v\|^2 + \eta^2 m^2 R^2$ because each $s_l(v \cdot z^{(l)}) \leq 0$ and $\|\Delta v\| \leq \eta m R$. The same inequality as above bounds the number of epochs.

Not separable. If training stopped, the final (w, θ) would classify all patterns correctly, so the sets would be separable. □

Remark. Training with the encoding false $= 0$, true $= 1$ cannot change w_i on an error where $x_i = 0$. ADALINE codes false as -1, so every weight moves on every error. The hidden units of a multi-layer network have no desired outputs, so the delta rule cannot train them; the solution of a task by hidden units is also not unique.

5 Multi-layer perceptrons

In the notes: Multi-Layer Perceptron

5.1 Layered networks

DEFINITION **r-layer perceptron** A feed-forward network whose units are partitioned into layers $U_1, \dots, U_r$ with $U_1 = U_{\text{in}}$, $U_r = U_{\text{out}}$ and connections only from U_l to U_{l+1}. Hidden and output units have the weighted sum as network input function. For layers of m and n units the weights form a matrix $W \in \mathbb{R}^{n \times m}$. With activation functions φ_l applied elementwise and thresholds θ_l, the layer outputs are

$$a_1 = x, \quad a_l = \varphi_l(W_l a_{l-1} - \theta_l), \quad l = 2, \dots, r, \tag{7}$$

and the network computes $y = a_r$. It has $\sum_{l=2}^{r} n_l (n_{l-1} + 1)$ parameters.

LEMMA **Linear hidden units collapse** If φ_l is the identity for $l = 2, \dots, r-1$, the network computes $y = \varphi_r(Mx - c)$ with $M = W_r W_{r-1} \cdots W_2$ and a fixed vector c, the same function as a network without hidden layers.

Proof. By induction, $a_l = W_l \cdots W_2 x - c_l$ for $l \le r-1$: the case $l = 2$ is the definition, and $a_{l+1} = W_{l+1} a_l - \theta_{l+1}$ is again affine in x. □

5.2 Universal approximation

THEOREM **Staircase approximation** Let $f : [a, b] \to \mathbb{R}$ be continuous and let $\sigma_k(z) = 1/\big(1 + e^{-kz}\big)$ be the logistic function with slope k. For every $\varepsilon > 0$ there are N, k, switching points $s_1 < \dots < s_N$ and weights $\Delta_1, \dots, \Delta_N$ such that the three-layer perceptron

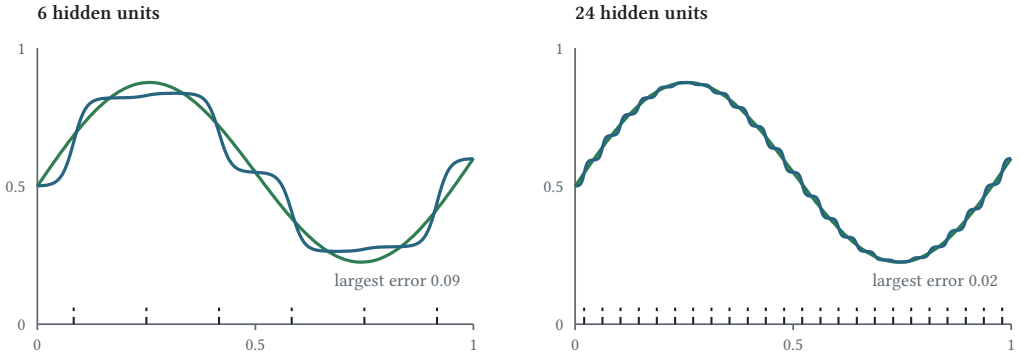
$$g(x) = f(a) + \sum_{i=1}^{N} \Delta_i \sigma_k(x - s_i) \tag{8}$$

satisfies $\sup_{x \in [a,b]} |f(x) - g(x)| \le 3\varepsilon$.

Proof. f is uniformly continuous, so there is $h > 0$ with $|f(x) - f(y)| < \varepsilon$ whenever $|x - y| \le h$. Take nodes $a = t_0 < \dots < t_N = b$ with $t_i - t_{i-1} \le h$, let $s_i = (t_{i-1} + t_i)/2$, $\Delta_i = f(t_i) - f(t_{i-1})$ and $V = \sum_i |\Delta_i|$. Let H be the unit step. The staircase $S(x) = f(a) + \sum_i \Delta_i H(x - s_i)$ equals $f\big(t_j\big)$ on $\big(s_j, s_{j+1}\big)$ because $\sum_{i \le j} \Delta_i = f\big(t_j\big) - f(a)$, and every x in that interval is within h of t_j, so $|f(x) - S(x)| < \varepsilon$ there.

Let $\eta = \min_i (s_{i+1} - s_i)/2$ and choose k so that $V/\big(1 + e^{k\eta}\big) < \varepsilon$. If x is at distance at least η from every s_i, each term satisfies $|\sigma_k(x - s_i) - H(x - s_i)| \le 1/\big(1 + e^{k\eta}\big)$, so $|g(x) - S(x)| < \varepsilon$ and $|f(x) - g(x)| < 2\varepsilon$. If x is within η of s_j (for one j only), the terms with $i \ne j$ again contribute an error below ε in total, and the jth term lies between 0 and Δ_j; so $g(x)$ lies within ε of the interval between $f\big(t_{j-1}\big)$ and $f\big(t_j\big)$, both within ε of $f(x)$ because x is within h of t_{j-1} and t_j. Hence $|f(x) - g(x)| \le 3\varepsilon$. □

Remark. The result extends to continuous functions of d variables, to any bounded non-constant sigmoid and to approximation of integrable functions in an L^p norm: Cybenko (1989) for sigmoids with limits 0 and 1 in the sup norm, Hornik (1991) for bounded non-constant activations. Leshno, Lin, Pinkus and Schocken (1993) prove that a continuous activation gives a universal one-hidden-layer family if and only if it is not a polynomial. The lectures obtain the multivariate case with bumps built from steps, which uses two hidden layers (four layers in their counting) and measures the error as the area between the functions. The staircase needs of the order of ε^{-d} pieces for a Lipschitz function of d variables; Barron (1993) gives a rate independent of d for functions with a bounded first Fourier moment. That some functions need exponentially many units in a shallow network but few in a deep one is shown by Telgarsky (2016) and Eldan and Shamir (2016).



Green: the function. Blue: the network output. Dashes at the bottom mark where each hidden unit switches.

Figure 3: The construction of the theorem with $N = 6$ and $N = 24$ hidden units and $k = 12N$ on $[0, 1]$: the largest error falls from 0.09 to 0.02.

6 Activation functions

In the notes: Activation Functions

DEFINITION Sigmoid function A function $\varphi : \mathbb{R} \to \mathbb{R}$ that is monotonically non-decreasing with finite limits at $-\infty$ and $+\infty$. Examples are the step, the semi-linear function (linear between two points, constant outside), the sine until saturation, the logistic function and the hyperbolic tangent.

DEFINITION Logistic function $\sigma(z) = 1/(1 + e^{-z})$, with values in $(0, 1)$ and $\sigma(0) = 1/2$.

LEMMA $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \le 1/4$, with equality at $z = 0$; $\sigma(-z) = 1 - \sigma(z)$; and $\tanh z = 2\sigma(2z) - 1$, $\tanh'(z) = 1 - \tanh^2 z \le 1$.

Proof. $\sigma' = e^{-z}/(1 + e^{-z})^2 = \sigma \cdot (e^{-z}/(1 + e^{-z})) = \sigma(1 - \sigma)$. The product of two numbers in $[0, 1]$ with sum 1 is at most $1/4$ (AM-GM), with equality for both equal to $1/2$. Further $2\sigma(2z) - 1 = (1 - e^{-2z})/(1 + e^{-2z}) = \tanh z$, so $\tanh' = 4\sigma'(2z) = 4\sigma(2z)(1 - \sigma(2z)) = (1 + \tanh z)(1 - \tanh z) = 1 - \tanh^2 z$. □

DEFINITION Rectified linear unit $\mathrm{relu}(z) = \max(0, z)$, with derivative 0 for $z < 0$ and 1 for $z > 0$ (undefined at 0). The leaky variant uses αz for $z < 0$; the softplus $\log(1 + e^z)$ is a smooth version, with derivative $\sigma(z)$.

Remark. The ReLU is unbounded, hence not a sigmoid, but it is not a polynomial and so gives universal one-hidden-layer networks (Leshno et al., 1993).

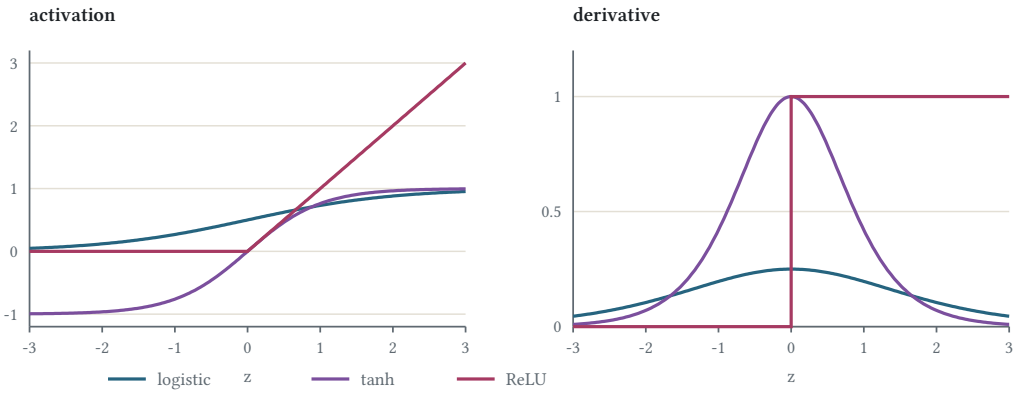


Figure 4: The logistic function, tanh and the ReLU (left) and their derivatives (right). The two sigmoids lose their slope for large $|z|$.

7 Training

In the notes: Training a Multi-Layer Perceptron

7.1 Gradient descent

DEFINITION Gradient descent For an error $e(w)$ differentiable in the parameters w (weights and thresholds), repeat $w \leftarrow w - \eta \nabla_w e(w)$ with learning rate $\eta > 0$. Online descent uses the gradient of one pattern's error, batch descent that of the sum over the training set.

LEMMA Logistic unit For $y = \sigma(w \cdot x - \theta)$ and the pattern error $e = (o - y)^2$,

$$\partial e / \partial w_i = -2(o - y)y(1 - y)x_i, \quad \partial e / \partial \theta = 2(o - y)y(1 - y). \tag{9}$$

Absorbing 2 into η, the corrections are $\Delta w_i = \eta(o - y)y(1 - y)x_i$ and $\Delta\theta = -\eta(o - y)y(1 - y)$.

Proof. The chain rule with $\partial y / \partial(w \cdot x - \theta) = y(1 - y)$ and $\partial(w \cdot x - \theta)/\partial w_i = x_i$, $\partial / \partial\theta = -1$. □

Remark. The factor $y(1 - y)$ is at most $1/4$ and vanishes as $y \to 0$ or 1: a saturated unit changes slowly whatever its error. For targets in $\{0, 1\}$ the error of a logistic output cannot reach 0, since $y \in (0, 1)$.

7.2 Variants

DEFINITION Momentum $\Delta w(t) = -\eta \nabla e(t) + \beta \Delta w(t - 1)$ with $0 \le \beta < 1$. In a region of constant gradient the steps converge to $-\eta \nabla e / (1 - \beta)$, an effective rate $1/(1 - \beta)$ times larger.

DEFINITION Resilient propagation Every parameter w_i has its own step size Δ_i. With $g_{i(t)} = \partial e / \partial w_i$ at step t:

$$\Delta_{i(t)} = \begin{cases} \min\left(a^+ \Delta_{i(t-1)}, \Delta_{\max}\right) & \text{if } g_{i(t)} g_{i(t-1)} > 0 \\ \max\left(a^- \Delta_{i(t-1)}, \Delta_{\min}\right) & \text{if } g_{i(t)} g_{i(t-1)} < 0, \\ \Delta_{i(t-1)} & \text{otherwise} \end{cases} \quad \Delta w_i = -\operatorname{sign}\left(g_{i(t)}\right) \Delta_{i(t)}, \tag{10}$$

with $a^+ = 1.2$ and $a^- = 0.5$ (Riedmiller and Braun, 1993). Only the sign of the gradient enters.

Remark. Manhattan training is $\Delta w = -\eta\,\operatorname{sign}(\nabla e)$ with a fixed step. *Flat-spot elimination* replaces φ' by $\varphi' + \zeta$ for a small constant ζ in the gradient. *Quickprop* (Fahlman, 1988) approximates e in each weight by a parabola through the last two gradients and jumps to its vertex, $\Delta w(t) = g(t)/(g(t-1) - g(t)) \cdot \Delta w(t-1)$.

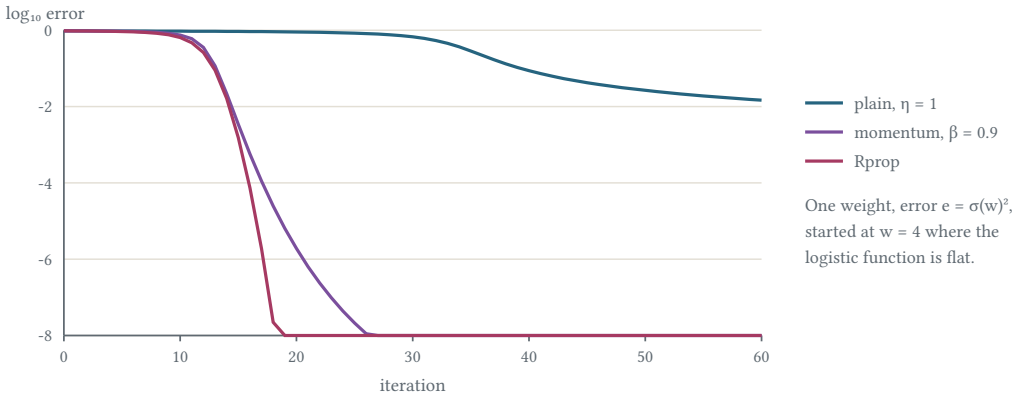


Figure 5: Log error against iteration for $e = \sigma(w)^2$ started at $w = 4$: plain gradient descent with $\eta = 1$ stays on the flat tail, momentum with $\beta = 0.9$ and Rprop leave it within about 30 iterations.

7.3 Complexity

DEFINITION **Weight decay** Minimize $e_{\text{tot}} = e + (\xi/2)\sum_i w_i^2$ with $\xi > 0$. The gradient step becomes $w_i \leftarrow w_i(1 - \eta\xi) - \eta\partial e/\partial w_i$, since $\partial/\partial w_i((\xi/2)w_i^2) = \xi w_i$. For a linear unit with squared error this is ridge regression.

DEFINITION **Early stopping** Evaluate the error on a validation set after each epoch and stop when it begins to increase, keeping the parameters of the epoch with the smallest validation error.

DEFINITION **Sensitivity analysis** The sensitivity of the outputs to input i over a training set L is

$$s_i = \frac{1}{|L|}\sum_{l \in L}\sum_{v \in U_{\text{out}}} |\partial\,\text{out}_v^{(l)}\,/\partial\,\text{ext}_i^{(l)}|, \tag{11}$$

computed by back-propagation to the input units. It depends on the scale of the inputs and, being local, can underrate inputs that share their effect with correlated ones.

8 Depth

In the notes: Deep Networks

THEOREM **Depth gives exponentially many pieces** Let $t(x) = 2\max(0, x) - 4\max(0, x - 1/2)$, so that $t(x) = 2x$ on $[0, 1/2]$ and $t(x) = 2 - 2x$ on $[1/2, 1]$. The k-fold composition t^k, computed by k layers of two ReLU units, is piecewise linear on $[0, 1]$ with 2^k pieces, each mapping onto $[0, 1]$. A network with one hidden layer of m ReLU units is piecewise linear on $\mathbb{R}$ with at most $m + 1$ pieces, so representing t^k on $[0, 1]$ takes $m \geq 2^k - 1$ units.

Proof. For $k = 1$ the two pieces map onto $[0, 1]$. If t^k has 2^k linear pieces each mapping affinely onto $[0, 1]$, then on each piece $t^{k+1} = t \circ t^k$ is the composition of an affine onto map with t, which has two linear pieces;

so t^{k+1} has 2^{k+1} pieces, again onto $[0,1]$. For the shallow network, $c+\sum_{i=1}^{m} a_i \max(0, w_i x - b_i)$ is linear except possibly at the m points b_i/w_i (a unit with $w_i = 0$ is constant), so it has at most $m+1$ pieces; t^k has 2^k pieces with distinct slopes on neighbouring pieces, hence needs $m+1 \geq 2^k$. □

Remark. Parity. The lectures give 2^{n-1} hidden units for the parity of n bits, which is the construction of one unit per input with output 1. A single hidden layer of n threshold units suffices: $h_k = [x_1 + \ldots + x_n \geq k]$ for $k = 1, \ldots, n$ and the output $y = \left[\sum_k (-1)^{k+1} h_k \geq 1/2\right]$. If s inputs are on, $h_1 = \ldots = h_s = 1$ and the others are 0, so the sum is 1 for odd s and 0 for even s. A chain of exclusive-or blocks of three threshold units each also gives $3(n-1)$ units in $n-1$ layers. Exponential separations of depth from width are proved for other functions: Telgarsky (2016) for the sawtooth above, Eldan and Shamir (2016) for a radial function in $\mathbb{R}^d$.

LEMMA Vanishing gradient bound In a layered network with $\delta_l = \partial e / \partial z_l$ the derivative with respect to the net input of layer l, $\delta_l = \left(W_{l+1}^{\top} \delta_{l+1}\right) \odot \varphi'(z_l)$. If $\|W_l\|_2 \leq w$ for all l and $|\varphi'| \leq c$, then $\|\delta_l\| \leq (cw)^{L-l} \|\delta_L\|$.

Proof. $\left\|W^{\top} v\right\| \leq \|W\|_2 \|v\|$ and a componentwise product with a vector bounded by c multiplies the norm by at most c; iterate from $l = L-1$ down. □

Remark. For the logistic function $c = 1/4$, so the bound decays unless $w > 4$; for tanh $c = 1$; for the ReLU $c = 1$ on active units.

DEFINITION Dropout In each training step every input or hidden unit is omitted independently with probability p, together with its connections. At test time all units are used and the outgoing weights are multiplied by $1-p$ (equivalently, activations are scaled by $1/(1-p)$ during training). See Srivastava et al. (2014).

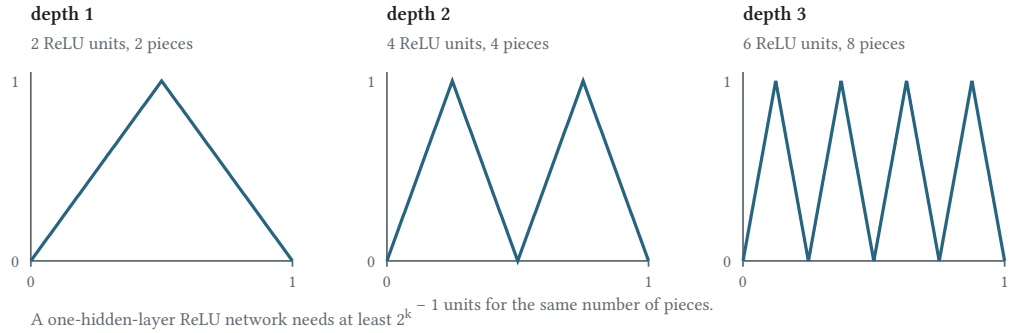


Figure 6: The tent function composed k times has 2^k linear pieces and needs $2k$ ReLU units.

9 Autoencoders

In the notes: Autoencoders

DEFINITION Autoencoder A three-layer perceptron with d inputs, k hidden units and d outputs trained on a free learning task $L = \left\{x^{(l)}\right\}$ with the inputs as desired outputs, that is by minimizing $\sum_l \left\|x^{(l)} - \hat{x}^{(l)}\right\|^2$. The hidden layer computes the code $f = \varphi(W_1 x - b_1)$ (the encoder) and the output layer the reconstruction $\hat{x} = \psi(W_2 f - b_2)$ (the decoder). Only the encoder is used afterwards.

THEOREM Linear autoencoder and principal components Let $X \in \mathbb{R}^{d\times N}$ hold centred data as columns, with singular value decomposition $X = U\Sigma V^\top$ and singular values $\sigma_1 \geq \sigma_2 \geq \ldots$, and let $k \leq \operatorname{rank}(X)$. For all $W_1 \in \mathbb{R}^{k\times d}$, $W_2 \in \mathbb{R}^{d\times k}$,

$$\|X - W_2 W_1 X\|_F^2 \geq \sum_{i>k} \sigma_i^2, \tag{12}$$

with equality for $W_2 = U_k, W_1 = U_k^\top$, where U_k holds the first k columns of U. The optimal reconstruction is the orthogonal projection on the span of the first k principal axes.

Proof. $W_2 W_1 X$ has rank at most k. By the Eckart-Young theorem the best approximation of X of rank at most k in the Frobenius norm is $U_k \Sigma_k V_k^\top$, with squared error $\sum_{i>k} \sigma_i^2$. Finally $U_k U_k^\top X = U_k \Sigma_k V_k^\top$. □

Remark. The optimal weights are not unique: for invertible $A \in \mathbb{R}^{k\times k}$, replacing (W_1, W_2) by $(AW_1, W_2 A^{-1})$ leaves $W_2 W_1$ unchanged. Only the span of the code is determined.

DEFINITION Denoising autoencoder An autoencoder whose input is a corrupted version $\tilde{x}$ of x (added noise, or a random subset of components set to zero) and whose desired output is the clean x.

DEFINITION Stacked autoencoder A network built in steps. Step 1 trains an autoencoder on the inputs and keeps the code $f^{(1)}$. Step j trains an autoencoder on $f^{(j-1)}$ and keeps $f^{(j)}$. A classifier or predictor is trained on the last code. The encoders are then stacked, the classifier added on top, and the whole network is fine-tuned by back-propagation.

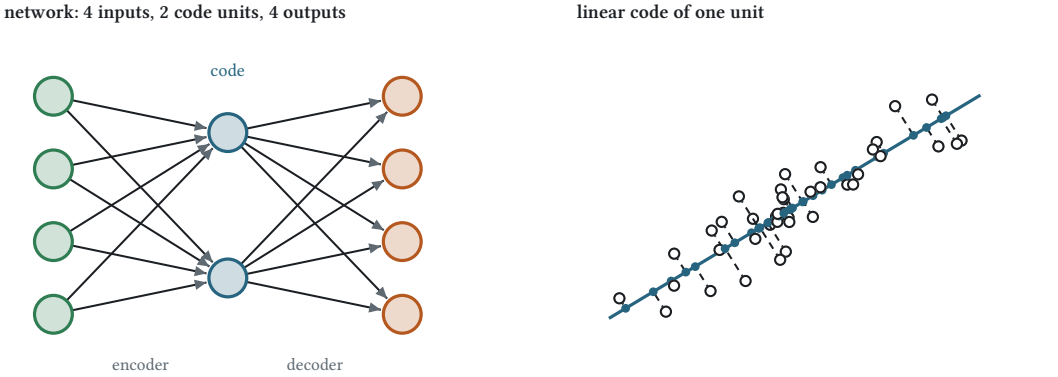


Figure 7: A 4-2-4 autoencoder (left); a linear autoencoder with a one-unit code on two-dimensional data (right): inputs (open) and their reconstructions on the first principal axis (filled), mean squared error 0.050.

10 Radial basis function networks

In the notes: Radial Basis Function Networks

DEFINITION Radial activation function A monotonically decreasing $f : [0, \infty) \to [0, 1]$ with $f(0) = 1$ and $\lim_{r\to\infty} f(r) = 0$. The Gaussian is $f(r) = e^{-r^2/2}$. The Minkowski distance is $d_p(x, y) = \left(\sum_i |x_i - y_i|^p\right)^{1/p}$, with $p = 1$ the city-block, $p = 2$ the Euclidean and $p \to \infty$ the maximum distance.

DEFINITION **Radial basis function network** A three-layer feed-forward network. Each hidden unit k has a centre c_k (its weight vector) and a radius $\sigma_k > 0$, net input $d(c_k, x)$ and activation $a_k = f(d(c_k, x)/\sigma_k)$. The output unit has the weighted sum as net input and the identity activation:

$$y(x) = \theta + \sum_{k=1}^{m} w_k a_k(x). \tag{13}$$

THEOREM **Interpolation** Let $\iota^{(1)}, \dots, \iota^{(m)}$ be distinct points, f Gaussian, d Euclidean and a common radius σ. Then the matrix $A \in \mathbb{R}^{m \times m}$ with $A_{lk} = f\big(d(\iota^{(k)}, \iota^{(l)})/\sigma\big)$ is positive definite, and for every $o \in \mathbb{R}^m$ the weights $w = A^{-1}o$ give a network with $\theta = 0$ and $y(\iota^{(l)}) = o_l$ for all l.

Proof. The Gaussian has the representation $e^{-\|x-y\|^2/2\sigma^2} = \int g(\omega) e^{i\omega\cdot(x-y)}\,\mathrm{d}\omega$ with $g > 0$ (its Fourier transform is a Gaussian). Hence for $a \neq 0$, $a^\top A a = \int g(\omega)\,|\sum_l a_l e^{i\omega\cdot\iota^{(l)}}|^2\,\mathrm{d}\omega \geq 0$, and it vanishes only if $\sum_l a_l e^{i\omega\cdot\iota^{(l)}} = 0$ for almost every ω, which is impossible because exponentials with distinct frequencies are linearly independent. A positive definite matrix is invertible (Micchelli, 1986). The lectures choose $\sigma = d_{\max}/\sqrt{2m}$ with $d_{\max}$ the largest distance between two patterns. □

THEOREM **Least squares for the output weights** Let $A \in \mathbb{R}^{m \times (k+1)}$ have a first column of ones (for θ) and further columns $a_j(\iota^{(l)})$ for $k < m$ centres, and have full column rank. Then $\|Aw - o\|^2$ is minimal exactly at $w = (A^\top A)^{-1} A^\top o = A^+ o$, the pseudo-inverse solution.

Proof. The gradient of $\|Aw - o\|^2$ is $2A^\top(Aw - o)$; it vanishes at the stated w, and the function is convex with Hessian $2A^\top A$ positive definite, so the stationary point is the unique minimum. □

LEMMA **Gradients of a Gaussian unit** For one output, $e = \sum_l (o_l - y_l)^2$ and $a_k = \exp(-d_k^2/2\sigma_k^2)$ with $d_k^2 = \|x - c_k\|^2$:

$$\begin{aligned} \partial e/\partial w_k &= -2\sum_l (o_l - y_l) a_k, \\ \partial e/\partial c_k &= -2\sum_l (o_l - y_l) w_k a_k (x - c_k)/\sigma_k^2, \\ \partial e/\partial \sigma_k &= -2\sum_l (o_l - y_l) w_k a_k d_k^2/\sigma_k^3. \end{aligned} \tag{14}$$

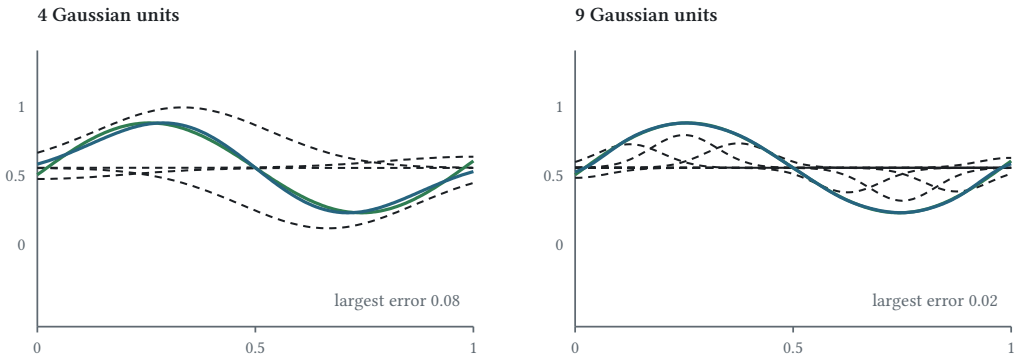
Proof. $\partial y/\partial w_k = a_k$, $\partial a_k/\partial c_k = a_k(x - c_k)/\sigma_k^2$ and $\partial a_k/\partial \sigma_k = a_k d_k^2/\sigma_k^3$, combined by the chain rule with $\partial e/\partial y = -2(o - y)$. □

DEFINITION **c-means** Given points $x_1, \dots, x_n$ and c, start from c centres; repeat (1) assign every point to its nearest centre, (2) replace each centre by the mean of its points, until the centres no longer change.

LEMMA The objective $J = \sum_j \min_i \left\|x_j - c_i\right\|^2$ does not increase in either step, so c-means stops after finitely many iterations.

Proof. Step (1) gives each point its nearest centre. For a group of n_i points with mean μ, $\sum \|x - c\|^2 = \sum \|x - \mu\|^2 + n_i\|\mu - c\|^2$, so replacing c by μ does not increase J. There are finitely many partitions and J strictly decreases unless the partition is unchanged, so the algorithm stops. □

The solution of the training task has up to three phases: (1) output weights by A^+o, (2) centres by clustering and then phase 1, (3) phase 2 and then gradient descent on all parameters with the lemma above.



Green: the function. Blue: the network. Dashes: the bias plus one weighted unit each. Weights by least squares.

Figure 8: Gaussian units on a regular grid with radius 0.6 times the spacing and least-squares output weights: largest error 0.08 with 4 units, 0.02 with 9.

11 Learning vector quantization

In the notes: Learning Vector Quantization

DEFINITION **Learning vector quantization** A two-layer network with reference (codebook) vectors $r_1, \dots, r_c$ as the weight vectors of the output units, a distance as network input and a radial activation. Each reference vector may carry a class. Training adapts the vectors on a set of patterns, as follows. For each pattern x find the winner r closest to x and update only it: attraction $r \leftarrow r + \eta(x - r)$ if the classes agree (or without classes), repulsion $r \leftarrow r - \eta(x - r)$ if they differ. The learning rate decreases over time, for instance $\eta(t) = \eta_0 \alpha^t$ with $0 < \alpha < 1$.

DEFINITION **Winner takes all** The output function of the network: $\mathrm{out}_u = 1$ if $\mathrm{act}_u = \max_v \mathrm{act}_v$ and 0 otherwise, ties broken at random.

Remark. Two-winner rule: if the two nearest vectors have different classes, attract the one with the pattern's class and repel the other. *Window rule:* apply it only if $\min(d_1/d_2, d_2/d_1) > (1-\xi)/(1+\xi)$ for the distances d_1, d_2 to those vectors, with window parameter ξ (Kohonen).

LEMMA **Online c-means** Without classes, if a vector is attracted with $\eta = 1/n$ at its nth winning pattern, it equals the mean of the patterns it has won.

Proof. By induction: $r_n = r_{n-1} + (x_n - r_{n-1})/n = ((n-1)r_{n-1} + x_n)/n$, and the mean of n patterns is $((n-1)\mu_{n-1} + x_n)/n$. □

DEFINITION **Expectation maximization for a Gaussian mixture** For the density $p(x) = \sum_c \pi_c \mathcal{N}(x; \mu_c, \Sigma_c)$ and data $x_1, \dots, x_n$: the E-step computes $\gamma_{jc} = \pi_c \mathcal{N}(x_j; \mu_c, \Sigma_c) / \sum_{c'} \pi_{c'} \mathcal{N}(x_j; \mu_{c'}, \Sigma_{c'})$, the probability that component c generated x_j; the M-step sets $\pi_c = 1/n \sum_j \gamma_{jc}$, $\mu_c = \sum_j \gamma_{jc} x_j / \sum_j \gamma_{jc}$ and $\Sigma_c = \sum_j \gamma_{jc} (x_j - \mu_c)(x_j - \mu_c)^\top / \sum_j \gamma_{jc}$. Soft LVQ is its online version.

THEOREM **EM does not decrease the likelihood** With θ the parameters and $\ell(\theta) = \sum_j \log \sum_c p(x_j, c \mid \theta)$, the EM update $\theta_{t+1} = \arg\max_\theta Q(\theta \mid \theta_t)$, with $Q(\theta \mid \theta_t) = \sum_j \sum_c \gamma_{jc}^{(t)} \log p(x_j, c \mid \theta)$, satisfies $\ell(\theta_{t+1}) \geq \ell(\theta_t)$.

Proof. With $\gamma_{jc}^{(t)} = p(x_j, c \mid \theta_t)/p(x_j \mid \theta_t)$, Jensen's inequality for the concave logarithm gives

$$\begin{aligned} \ell(\theta) - \ell(\theta_t) &= \sum_j \log \sum_c \gamma_{jc}^{(t)} \frac{p(x_j, c \mid \theta)}{\gamma_{jc}^{(t)} p(x_j \mid \theta_t)} \\ &\geq \sum_j \sum_c \gamma_{jc}^{(t)} \log \frac{p(x_j, c \mid \theta)}{p(x_j, c \mid \theta_t)} \\ &= Q(\theta \mid \theta_t) - Q(\theta_t \mid \theta_t). \end{aligned} \tag{15}$$

Taking $\theta = \theta_{t+1}$ makes the right side nonnegative. □

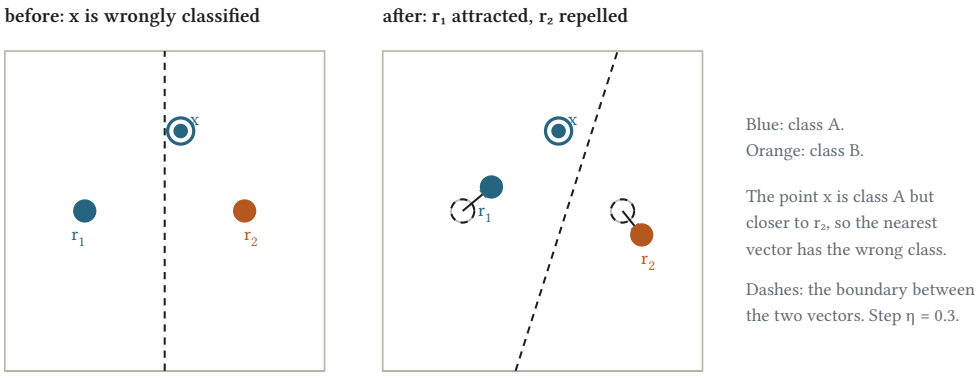


Figure 9: One LVQ step with $\eta = 0.3$: the winner r_2 has the wrong class and is repelled, the nearest vector r_1 of the right class is attracted, and the boundary moves across the pattern x.

12 Self-organizing maps

In the notes: Self-Organizing Maps

DEFINITION **Self-organizing map** A two-layer network like an LVQ network whose output units lie on a grid (usually two-dimensional, square or hexagonal) with a distance d_{grid} between units. Unit u has a reference vector r_u in the data space. The map assigns an input to the winner u^* whose reference vector is nearest.

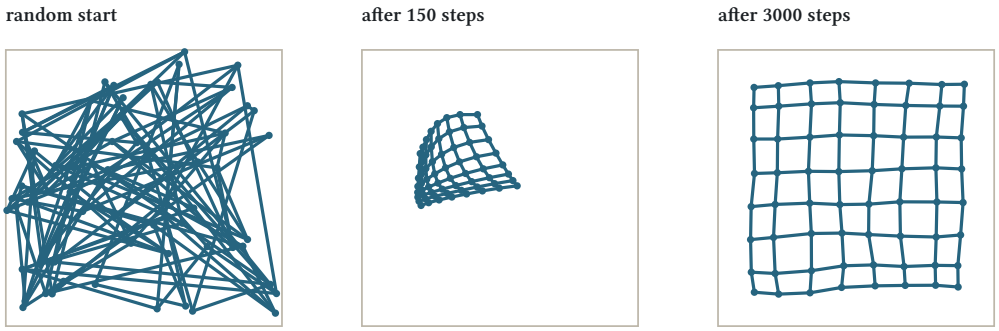
DEFINITION **Neighbourhood function** A radial function $f(d, \rho)$ of the grid distance, decreasing in d with radius ρ; for example $f(d, \rho) = e^{-d^2/2\rho^2}$. Training updates all units:

$$r_u \leftarrow r_u + \eta(t)\, f\big(d_{\text{grid}(u,u^*)}, \rho(t)\big)\,(x - r_u), \tag{16}$$

with learning rate $\eta(t)$ and radius $\rho(t)$ decreasing over time, for instance geometrically, $\eta(t) = \eta_0(\eta_T/\eta_0)^{t/T}$ and $\rho(t) = \rho_0(\rho_T/\rho_0)^{t/T}$.

Remark. For $\rho \to 0$ only the winner is updated and the rule becomes the attraction rule of LVQ. A step function as f gives a sharp neighbourhood, a smooth f shrinks it gradually. Training can fail, leaving a twisted map, if the initial η or ρ is too small. The map preserves topology (neighbours in the data map to

neighbours on the grid) but not distances, and reference vectors at the border lie slightly inside the data because the border units are pulled from one side only.



Each dot is a reference vector, lines join neighbours on the grid. The inputs are uniform in the square.

Figure 10: An 8×8 map trained on uniform points of the unit square (η from 0.6 to 0.02, ρ from 4 to 0.6 over 3000 steps, Gaussian neighbourhood): the random start contracts to a patch and unfolds over the square.

13 Summary of formulas

QUANTITY	RESULT	WHERE EXPLAINED
Network input, weighted sum	$\mathrm{net}_u = \sum_{v \in \mathrm{pred}(u)} w_{uv} \, \mathrm{out}_v$	Introduction
Unit state	$\mathrm{net}_u \to \mathrm{act}_u = f_{\mathrm{act}(\mathrm{net}_u, \theta_u)} \to \mathrm{out}_u = f_{\mathrm{out}(\mathrm{act}_u)}$	Introduction
Feed-forward evaluation	one pass in topological order, $\lvert U \rvert$ unit evaluations	Introduction
Sum of squared errors	$e = \sum_l \sum_k \left(o_k^{(l)} - \mathrm{out}_{v_k}^{(l)} \right)^2$	Introduction
Gaussian log-likelihood	$\ell = -e/(2\sigma^2) - (Nm/2) \log(2\pi\sigma^2)$	Introduction
Standardization	$x' = (x - \bar{x})/s$, mean 0, deviation 1	Introduction
One-hot distance	$\lVert e_i - e_j \rVert = \sqrt{2}$	Introduction
TLU	$y = 1 \Leftrightarrow w \cdot x \geq \theta$	Threshold Logic Units
Separability	$\mathrm{conv}(A_0) \cap \mathrm{conv}(A_1) = \varnothing$	Threshold Logic Units
Unit firing only at a	$w_i = 2a_i - 1, \theta = \sum_i a_i$	Threshold Logic Units
Delta rule	$\Delta\theta = -\eta(o - y), \Delta w_i = \eta(o - y)x_i$	Threshold Logic Units
Corrections, online, $v_0 = 0$	$k \leq (R\lVert v^* \rVert/\gamma)^2$	Threshold Logic Units
Layer	$a_l = \varphi_l(W_l a_{l-1} - \theta_l)$	Multi-Layer Perceptron
Parameters of an MLP	$\sum_l n_l(n_{l-1} + 1)$	Multi-Layer Perceptron
Staircase network	$g = f(a) + \sum_i \Delta_i \sigma_k(x - s_i), \quad \Delta_i = f(t_i) - f(t_{i-1})$	Multi-Layer Perceptron
Logistic slope	$\sigma' = \sigma(1 - \sigma) \leq 1/4; \quad \tanh' = 1 - \tanh^2$	Activation Functions
Logistic unit step	$\Delta w_i = \eta(o - y)y(1 - y)x_i$	Training a Multi-Layer Perceptron
Momentum	$\Delta w(t) = -\eta \nabla e(t) + \beta \Delta w(t - 1)$	Training a Multi-Layer Perceptron
Weight decay	$w \leftarrow w(1 - \eta\xi) - \eta \partial e/\partial w$	Training a Multi-Layer Perceptron
Sensitivity	$s_i = \lvert L \rvert^{-1} \sum_l \sum_v \lvert \partial \, \mathrm{out}_v / \partial \, \mathrm{ext}_i \rvert$	Training a Multi-Layer Perceptron

QUANTITY	RESULT	WHERE EXPLAINED
Tent composition	2^k pieces from $2k$ ReLU units; shallow net needs $\geq 2^k - 1$	Deep Networks
Gradient bound	$\|\delta_l\| \leq (cw)^{L-l}\|\delta_L\|$	Deep Networks
Linear autoencoder	error $= \sum_{i>k} \sigma_i^2$ at the projection on U_k	Autoencoders
RBF network	$y = \theta + \sum_k w_k f(d(c_k, x)/\sigma_k)$	Radial Basis Function Networks
Output weights	$w = (A^\top A)^{-1} A^\top o$	Radial Basis Function Networks
LVQ updates	$r \leftarrow r \pm \eta(x - r)$	Learning Vector Quantization
SOM update	$r_u \leftarrow r_u + \eta\, f(d_{\text{grid}}, \rho)(x - r_u)$	Self-Organizing Maps

14 References

- R. Kruse, C. Borgelt, F. Klawonn, C. Moewes, M. Steinbrecher and P. Held, *Computational Intelligence: A Methodological Introduction*, Springer, 2013: the source of the general definition of a network (units, the three functions, input and output units) and of the structure of the lectures; Part I on neural networks.
- S. Haykin, *Neural Networks and Learning Machines*, 3rd ed., Pearson, 2009: the definition of a neural network as an adaptive parallel processor, and the properties usually listed for it.
- W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity", *Bulletin of Mathematical Biophysics* 5 (1943), 115 to 133; F. Rosenblatt, "The perceptron: a probabilistic model for information storage and organization in the brain", *Psychological Review* 65 (1958), 386 to 408.
- F. A. C. Azevedo et al., "Equal numbers of neuronal and nonneuronal cells make the human brain an isometrically scaled-up primate brain", *Journal of Comparative Neurology* 513 (2009), 532 to 541: the count of about 86 billion neurons.
- A. S. Morcos, D. G. T. Barrett, N. C. Rabinowitz and M. Botvinick, "On the importance of single directions for generalization", *ICLR* 2018.
- B. Widrow and M. E. Hoff, "Adaptive switching circuits", *IRE WESCON Convention Record* 4 (1960), 96 to 104: the delta rule and ADALINE.
- A. B. J. Novikoff, "On convergence proofs on perceptrons", *Symposium on the Mathematical Theory of Automata* 12 (1962), 615 to 622: the convergence argument above. M. Minsky and S. Papert, *Perceptrons*, MIT Press, 1969: the limits of a single unit.
- OEIS A000609: the number of threshold functions of n variables.
- G. Cybenko, "Approximation by superpositions of a sigmoidal function", *Mathematics of Control, Signals and Systems* 2 (1989), 303 to 314; K. Hornik, "Approximation capabilities of multilayer feedforward networks", *Neural Networks* 4 (1991), 251 to 257; M. Leshno, V. Y. Lin, A. Pinkus and S. Schocken, "Multilayer feedforward networks with a nonpolynomial activation function can approximate any function", *Neural Networks* 6 (1993), 861 to 867: universal approximation.
- A. R. Barron, "Universal approximation bounds for superpositions of a sigmoidal function", *IEEE Transactions on Information Theory* 39 (1993), 930 to 945; M. Telgarsky, "Benefits of depth in neural networks", *COLT* 2016; R. Eldan and O. Shamir, "The power of depth for feedforward neural networks", *COLT* 2016: how many units are needed.
- M. Riedmiller and H. Braun, "A direct adaptive method for faster backpropagation learning: the RPROP algorithm", *IEEE ICNN* 1993; S. E. Fahlman, "An empirical study of learning speed in back-propagation networks", CMU technical report, 1988: the training variants.
- N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting", *Journal of Machine Learning Research* 15 (2014), 1929 to 1958.
- J. Park and I. W. Sandberg, "Universal approximation using radial-basis-function networks", *Neural Computation* 3 (1991), 246 to 257; C. A. Micchelli, "Interpolation of scattered data: distance matrices and conditionally positive definite functions", *Constructive Approximation* 2 (1986), 11 to 22.

- T. Kohonen, *Self-Organizing Maps*, 3rd ed., Springer, 2001: LVQ, the two-winner and window rules, and the map; A. P. Dempster, N. M. Laird and D. B. Rubin, “Maximum likelihood from incomplete data via the EM algorithm”, *Journal of the Royal Statistical Society B* 39 (1977), 1 to 38.
- C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, section 5.2: the error function of a network from maximum likelihood.
- Y. LeCun, L. Bottou, G. B. Orr and K.-R. Müller, “Efficient BackProp”, in *Neural Networks: Tricks of the Trade*, Springer, 1998: preparation of the inputs.